

Verina Specification Feedback

We are writing to share some observations regarding the quality of the problem specifications in Verina, and we hope this feedback may be useful for improving the benchmark. We would like to emphasize that our intent is entirely constructive, and we greatly appreciate the effort put into building Verina.

Our focus is on the specifications themselves rather than on whether any particular implementation conforms to a given specification. The quality of specifications is crucial, because it determines whether proofs about code are meaningful. In extreme cases, if a postcondition is simply `True`, then any Lean program that passes type checking would verify successfully — yet this would not provide any real correctness guarantee.

Of course, in most cases the postconditions are far from trivially `True`, and specifications with minor gaps may still provide useful correctness guarantees for the individual function. However, even subtle underspecification can become problematic in the context of larger systems, where higher-level proofs may rely on precise postcondition contracts as building blocks. As a concrete example, consider `basic_97`, which requires updating the `j`-th element of an array `a` to 60 while leaving all other elements unchanged. The postcondition provided by Verina is $(\text{result}[j]! = 60) \wedge (\forall k, k < \text{a.size} \rightarrow k \neq j \rightarrow \text{result}[k]! = \text{a}[k]!)$. This postcondition, however, does not enforce `result.size = a.size`, meaning that an implementation which sets `a[j]` to 60 and then appends arbitrary extra elements would still be accepted as correct under this specification.

In this report, we have identified 16 potential specification issues in the Verina benchmark where the specifications appear to be either underspecified or possibly incorrect. We hope the following overview and detailed analysis are helpful.

Problem ID	Potential Issue
Verina_advanced_8	Underspecified Post-condition
Verina_advanced_10	Unsatisfiable Specification
Verina_advanced_12	Incorrect Post-condition
Verina_advanced_13	Incorrect Post-condition
Verina_advanced_47	Underspecified Post-condition
Verina_advanced_71	Underspecified Post-condition
Verina_basic_22	Underspecified Post-condition
Verina_basic_34	Underspecified Post-condition
Verina_basic_37	Underspecified Post-condition
Verina_basic_46	Underspecified Post-condition
Verina_basic_77	Underspecified Post-condition
Verina_basic_79	Incorrect Post-condition
Verina_basic_84	Underspecified Post-condition
Verina_basic_95	Underspecified Post-condition
Verina_basic_97	Underspecified Post-condition
Verina_basic_103	Underspecified Post-condition

The observations in this report are based on the following version, which was the latest available at the time of writing¹:

Various fix (#15) · sunblaze-ucb/verina@643ed3c

Verina_advanced_8

Problem Description

Description

This task requires writing a Lean 4 method that determines whether it is possible to complete a circular journey around a set of gas stations. Each gas station provides a certain amount of gas, and traveling from one station to the next consumes a certain amount of gas.

You start the journey at one of the gas stations with an empty tank. The goal is to find the starting station's index that allows completing the entire circuit once in the clockwise direction without running out of gas. If such a station exists, return its index. Otherwise, return -1.

If multiple solutions exist, return the one with the smallest starting gas station index.

Input

The input consists of two arrays of equal non-zero length:

¹This version likely incorporates the findings by Harmonic as of December 3, 2025.

gas: An array of integers where $\text{gas}[i]$ represents the amount of gas available at the i th station.

cost: An array of integers where $\text{cost}[i]$ is the amount of gas required to travel from station i to station $i + 1$.

Output

The output is an integer: Returns the index of the starting gas station that allows a complete trip around the circuit. If it is not possible to complete the circuit, return -1.

Verina's spec

```
@[reducible]
def canCompleteCircuit_precond (gas : List Int) (cost : List Int) : Prop :=
  -- !benchmark @start precondition
  gas.length > 0 ∧ gas.length = cost.length

-- !benchmark @end precondition

@[reducible]
def canCompleteCircuit_postcond (gas : List Int) (cost : List Int) (result: Int)
  ⇨ (h_precond : canCompleteCircuit_precond (gas) (cost)) : Prop :=
  -- !benchmark @start postcondition
  let valid (start : Nat) := List.range gas.length |>.all (fun i =>
    let acc := List.range (i + 1) |>.foldl (fun t j =>
      let jdx := (start + j) % gas.length
      t + gas[jdx]! - cost[jdx]!) 0
    acc ≥ 0)
  -- For result = -1: It's impossible to complete the circuit starting from any
  ⇨ index
  -- In other words, there's no starting point from which we can always maintain a
  ⇨ non-negative gas tank
  (result = -1 → (List.range gas.length).all (fun start => ¬ valid start)) ∧
  -- For result ≥ 0: This is the valid starting point
  -- When starting from this index, the gas tank never becomes negative during the
  ⇨ entire circuit
  (result ≥ 0 → result < gas.length ∧ valid result.toNat ∧ (List.range
    ⇨ result.toNat).all (fun start => ¬ valid start))

-- !benchmark @end postcondition
```

Observation

The post-condition appears to be underspecified.

It only constrains the behavior when $\text{result} = -1$ or $\text{result} \geq 0$, and therefore admits vacuous satisfaction for any $\text{result} \leq -2$. As a result, the post-condition does not rule out invalid return values, and may not fully characterize the intended output domain.

Verina_advanced_10

Problem Description

Description

This task requires writing a Lean 4 method that decomposes a natural number n into its prime factorization components based on a user-provided list of primes. Specifically, it calculates the exponents for each prime in the factorization such that: $n = \prod p^e$. In other words, it determines the exponent e for each prime p .

Input

The input consists of:

- n : A positive natural number ($n > 0$) that can be expressed as a product of powers of the given primes.
- primes : A non-empty list of distinct prime numbers whose product of powers equals n .

Output

The output is `List (Nat × Nat)`: Return a list of pair/Cartesian product of two natural numbers (p, e) , where p is the prime and e is the exponent of p in the factorization. Each prime in the output must be from the input list, and every prime in the input list must appear in the output.

Verina's Spec

```
@[reducible]
def findExponents_precond (n : Nat) (primes : List Nat) : Prop :=
  -- !benchmark @start precondition
  n > 0 ∧
  primes.length > 0 ∧
  primes.all (fun p => Nat.Prime p) ∧
  List.Nodup primes

  -- !benchmark @end precondition

@[reducible]
def findExponents_postcond (n : Nat) (primes : List Nat) (result: List (Nat ×
  ↪ Nat)) (h_precond : findExponents_precond (n) (primes)) : Prop :=
  -- !benchmark @start postcondition
  (n = result.foldl (fun acc (p, e) => acc * p ^ e) 1) ∧
  result.all (fun (p, _) => p ∈ primes) ∧
  primes.all (fun p => result.any (fun pair => pair.1 = p))
```

Observation

A potential concern with this specification is that the precondition does not require the input `primes` list to contain all prime factors of n , yet the postcondition enforces that the product of the primes raised to the output exponents

equals n . As a result, the specification may be unsatisfiable for any n whose prime factorization includes a prime not listed in `primes`.

Verina_advanced_12

Problem Description

Description

Write a Lean 4 function that returns the first duplicate integer found in a list. The function should return the value of the first duplicate it encounters, scanning from left to right. If no duplicates exist, return -1.

Input

lst: A list of integers.

Output

An Option Int:

- `some x` where x is the first duplicated value encountered when scanning left to right
- `none` if no duplicates exist in the list

Verina's spec

```
@[reducible]
def firstDuplicate_precond (lst : List Int) : Prop :=
  -- !benchmark @start precondition
  True

-- !benchmark @end precondition

@[reducible]
def firstDuplicate_postcond (lst : List Int) (result: Option Int) (h_precond :
  ↔ firstDuplicate_precond (lst)) : Prop :=
  -- !benchmark @start postcondition
  match result with
  | none => List.Nodup lst
  | some x =>
    lst.count x > 1 ∧
    (lst.filter (fun y => lst.count y > 1)).head? = some x

-- !benchmark @end postcondition
```

Observation

The specification may not correctly capture the intended semantics: it characterizes the first value with multiple occurrences in the list, rather than the first duplicate encountered when scanning left to right. For example, given the input

```
[1, 2, 2, 1]
```

the correct result should be `some 2`, but the specification only accepts `some 1`.

Moreover, the benchmark includes a reference implementation:

```
def firstDuplicate (lst : List Int) (h_precond : firstDuplicate_precond (lst)) :
  ⇔ Option Int :=
  let rec helper (seen : List Int) (rem : List Int) : Option Int :=
    match rem with
    | [] => none
    | h :: t => if seen.contains h then some h else helper (h :: seen) t
  helper [] lst
```

While this implementation correctly returns the first duplicate when scanning left to right, it appears to diverge from the specification.

Verina_advanced_13

Problem Description

Description

This task requires writing a Lean 4 method that determines whether there are any intersections between chords on a circle. The method should return true if at least one pair of chords intersects, and false otherwise.

A chord is defined as a line segment connecting two distinct points on a circle. Two chords intersect if they cross each other inside the circle. The points are numbered from 1 to $2N$ in a clockwise direction, where N is the number of chords.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq A_i, B_i \leq 2N$
- $A_1, \dots, A_N, B_1, \dots, B_N$ are all distinct
- All input values are integers

Input

The input consists of two parameters:

- N : A natural number representing the number of chords ($2 \leq N \leq 2 \times 10^5$).
- `chords`: A list of N pairs of natural numbers, where each pair represents the endpoints of a chord. All endpoint values are distinct and range from 1 to $2N$.

Output

The output is a boolean value:

- Returns true if there exists at least one pair of intersecting chords.
- Returns false if no chords intersect.

Verina's Spec

```

def hasChordIntersection_precond (N : Nat) (chords : List (List Nat)) : Prop :=
  -- !benchmark @start precondition
  N ≥ 2 ∧
  chords.all (fun chord => chord.length = 2 ∧ chord[0]! ≥ 1 ∧ chord[0]! ≤ 2 * N
    ↔ ∧ chord[1]! ≥ 1 ∧ chord[1]! ≤ 2 * N) ∧
  List.Nodup (chords.flatMap id)

-- !benchmark @end precondition

@[reducible]
def hasChordIntersection_postcond (N : Nat) (chords : List (List Nat)) (result :
  Bool) (h_precond : hasChordIntersection_precond (N) (chords)) : Prop :=
  -- !benchmark @start postcond
  let sortedChords := chords.map (fun chord =>
    let a := chord[0]!
    let b := chord[1]!
    if a > b then [b, a] else [a, b]
  )

  let rec hasIntersection (chord1 : List Nat) (chord2 : List Nat) : Bool :=
    let a1 := chord1[0]!
    let b1 := chord1[1]!
    let a2 := chord2[0]!
    let b2 := chord2[1]!
    (a1 < a2 && a2 < b1 && b1 < b2) || (a2 < a1 && a1 < b2 && b2 < b1)

  let rec checkAllPairs (chords : List (List Nat)) : Bool :=
    match chords with
    | [] => false
    | x :: xs =>
      if xs.any (fun y => hasIntersection x y) then true
      else checkAllPairs xs

  ((List.Pairwise (fun x y => !hasIntersection x y) sortedChords) → ¬ result) ∧
  ((sortedChords.any (fun x => chords.any (fun y => hasIntersection x y))) →
    result)

```

Observation

We noticed two points in this specification that may warrant attention.

First, the precondition does not enforce that the input contains exactly N chords. While the problem statement requires exactly N pairs of endpoints, the precondition only constrains endpoint validity and distinctness, leaving the cardinality of `chords` unconstrained. This appears to be a mismatch between the specification and the intended input domain.

Secondly, the postcondition appears to inconsistently mix normalized and non-normalized chord representations. In the last line,

```

((sortedChords.any (fun x => chords.any (fun y => hasIntersection x y))) →
  result)

```

the second quantification ranges over the original, potentially unsorted chords, even though `hasIntersection` assumes sorted endpoints. It may be more appropriate to have this range over `sortedChords` instead:

```
((sortedChords.any (fun x => sortedChords.any (fun y => hasIntersection x y))) →
↪ result)
```

As a counterexample that illustrates this issue, consider:

```
N = 2
chords = [[3, 1], [4, 2]]
```

For this input, the two chords do intersect, so the correct return value is `true`. However, because the second quantification ranges over the unsorted `chords`, `hasIntersection` never evaluates to `true` for any pair (since the unsorted endpoints violate its sorted assumption). As a result, the antecedents of both implications in the postcondition are `false`, making the entire postcondition vacuously satisfied for any return value — meaning both `true` and `false` would be accepted, and the postcondition may not enforce the intended answer.

Verina_advanced_47

Problem Description

Description

This task requires writing a Lean 4 method that merges all overlapping intervals from a given list of intervals.

Each interval is represented as a pair `[start, end]`, indicating the start and end of the interval. If two intervals overlap, they should be merged into a single interval that spans from the minimum start to the maximum end of the overlapping intervals.

The goal is to return a list of non-overlapping intervals that cover all the input intervals after merging.

Input

The input consists of one array:

intervals: An array of pairs of integers where `intervals[i] = [starti, endi]` represents the start and end of the *i*th interval.

Output

The output is an array of pairs of integers: Returns the list of merged, non-overlapping intervals sorted by their start times.

Verina's Spec

```
def mergeIntervals_precond (intervals : List (Prod Int Int)) : Prop :=
  -- !benchmark @start precondition
  True
```



```

-- !benchmark @end precondition

def mergeIntervals_postcond (intervals : List (Prod Int Int)) (result: List (Prod
↪ Int Int)) (h_precond : mergeIntervals_precond (intervals)) : Prop :=
  -- !benchmark @start postcond
  -- Check that all original intervals are covered by some result interval
  let covered := intervals.all (fun (s, e) =>
    result.any (fun (rs, re) => rs ≤ s ∧ e ≤ re))

  -- Check that no intervals in the result overlap
  let rec noOverlap (l : List (Prod Int Int)) : Bool :=
    match l with
    | [] | [_] => true
    | (_, e1) :: (s2, e2) :: rest => e1 < s2 && noOverlap ((s2, e2) :: rest)

  covered ∧ noOverlap result

-- !benchmark @end postcond

```

Observation

Both the precondition and postcondition appear to be underspecified.

First, the precondition is simply `True`, imposing no restrictions on the input. For interval problems, it is generally standard to require that the start of each interval is less than or equal to its end.

More importantly, the postcondition only ensures that each original interval is covered by some result interval and that the result intervals are non-overlapping. It does not require that the union of the result intervals exactly matches the union of the input intervals, which means a result that covers additional points beyond those in the input could still satisfy the specification.

Verina_advanced_71

Problem Description

Description

This task requires writing a Lean 4 method that, given a binary string `s` and an integer `k`, finds the shortest contiguous substring that contains exactly `k` characters '1'.

Among all substrings of `s` that contain exactly `k` occurrences of '1', return the one that is shortest in length. If there are multiple such substrings with the same length, return the lexicographically smallest one. If no such substring exists, return the empty string.

Input

- `s`: A binary string (only consisting of characters '0' and '1').

- k : A natural number ($k \geq 0$).

Output

A string representing the shortest substring of s that contains exactly k ones. If no such substring exists, return "".

Verina's Spec

```
@[reducible]
def shortestBeautifulSubstring_precond (s : String) (k : Nat) : Prop :=
  -- !benchmark @start precondition
  s.toList.all (fun c => c = '0' ∨ c = '1')

  -- !benchmark @end precondition

def shortestBeautifulSubstring_postcond (s : String) (k : Nat) (result: String)
↪ (h_precond : shortestBeautifulSubstring_precond (s) (k)) : Prop :=
  -- !benchmark @start postcondition
  let chars := s.data
  let substrings := (List.range chars.length).flatMap (fun i =>
    (List.range (chars.length - i + 1)).map (fun len =>
      chars.drop i |>.take len))
  let isBeautiful := fun sub => countOnes sub = k
  let beautiful := substrings.filter (fun sub => isBeautiful sub)
  let targets := beautiful.map (·.asString) |>.filter (fun s => s ≠ "")
  (result = "" ∧ targets = []) ∨
  (result ∈ targets ∧
   ∀ r ∈ targets, r.length ≥ result.length ∨ (r.length = result.length ∧ result
   ↪ ≤ r))

  -- !benchmark @end postcondition
```

Observation

Verina's post-condition appears to be underspecified.

The post-condition requires that for all r in **targets** (the set of all contiguous substrings of s containing exactly k characters '1'),

$$r.length \geq result.length \vee (r.length = result.length \wedge result \leq r).$$

This condition is intended to enforce that **result** is the shortest such substring and, among all shortest ones, the lexicographically smallest. However, the disjunction admits $r.length = result.length$ through the left branch $r.length \geq result.length$ without requiring $result \leq r$. Consequently, when there are multiple shortest substrings, any one of them satisfies the post-condition, not necessarily the lexicographically smallest one.

As a concrete example, consider the input $s = "11011"$ and $k = 3$. Both "1101" and "1011" contain exactly three '1' characters and have the same length. The correct answer is "1011" (lexicographically smaller), but **result** = "1101" also satisfies the post-condition because for $r = "1011"$ the left branch $r.length \geq$

`result.length` holds (both have length 4), making the disjunction true without checking the lexicographic order.

The fix is to replace `r.length ≥ result.length` with `r.length > result.length`, so that equal-length candidates are always compared lexicographically:

$\forall r \in \text{targets}, r.\text{length} > \text{result.length} \vee (r.\text{length} = \text{result.length} \wedge \text{result} \leq r).$

Verina__basic__22

Problem Description

Description

This task requires writing a Lean 4 method that identifies the dissimilar elements between two arrays of integers. In other words, the method should return an array containing all elements that appear in one input array but not in the other. The output array must contain no duplicate elements and the order of elements does not matter.

Input

The input consists of:

- a: An array of integers.
- b: An array of integers.

Output

The output is an array of integers: Returns an array containing all distinct elements from both input arrays that are not present in the other array and should be sorted

Verina's Spec

```
def inArray (a : Array Int) (x : Int) : Bool :=
  a.any (fun y => y = x)

def dissimilarElements_precond (a : Array Int) (b : Array Int) : Prop :=
  -- !benchmark @start precondition
  True

-- !benchmark @end precondition

def dissimilarElements_postcond (a : Array Int) (b : Array Int) (result: Array
  Int) (h_precond : dissimilarElements_precond (a) (b)) :=
  -- !benchmark @start postcondition
  result.all (fun x => inArray a x ≠ inArray b x) ∧
  result.toList.Pairwise (· ≤ ·) ∧
  a.all (fun x => if x ∈ b then x ∉ result else x ∈ result) ∧
  b.all (fun x => if x ∈ a then x ∉ result else x ∈ result)
```

```
-- !benchmark @end postcond
```

Observation

The post-condition appears to be underspecified.

The problem description requires that the output array contain no duplicate elements, but this constraint does not appear to be captured in the post-condition.

Additionally, we noticed a potential inconsistency within the problem description itself: the Description section states that the order of elements does not matter, whereas the Output section explicitly requires the output to be sorted. These two statements appear to be in tension and might benefit from clarification.

Verina__basic__34

Problem Description

Description

This task requires writing a Lean 4 method that extracts even numbers from an array of integers. The method should return a new array containing only the even numbers found in the input array, while preserving the order in which they appear.

Input

The input consists of: `arr`: An array of integers.

Output

The output is an array of integers: Returns an array containing all the even numbers from the input array. Specifically:

- Every element in the output array is an even integer.
- All even integers present in the input array are included in the output array.
- The relative order of the even integers is preserved as in the input array.

Note

There are no preconditions for this task; the method will work with any array, including empty arrays (which are not null).

Verina's Spec

```
def isEven (n : Int) : Bool :=
  n % 2 = 0

def findEvenNumbers_precond (arr : Array Int) : Prop :=
  -- !benchmark @start precondition
  True
```

```

-- !benchmark @end precondition

def findEvenNumbers_postcond (arr : Array Int) (result: Array Int) (h_precond :
↪ findEvenNumbers_precond (arr)) :=
  -- !benchmark @start postcond
  (∀ x, x ∈ result → isEven x ∧ x ∈ arr.toList) ∧
  (∀ x, x ∈ arr.toList → isEven x → x ∈ result) ∧
  (∀ x y, x ∈ arr.toList → y ∈ arr.toList →
    isEven x → isEven y →
    arr.toList.indexOf x ≤ arr.toList.indexOf y →
    result.toList.indexOf x ≤ result.toList.indexOf y)

-- !benchmark @end postcond

```

Observation

The post-condition appears to be underspecified.

It states that: 1. Any even number occurring in the input array should occur in the output array. 2. Any even number occurring in the output array should occur in the input array. 3. The relative order of elements is preserved, i.e., if x appears before y in the input, then x also appears before y in the output.

However, this specification does not account for duplicate elements. For example, given the input array $[2, 2]$, the output $[2]$ still satisfies all three conditions, even though one occurrence of 2 has been omitted.

Verina__basic__37

Problem Description

Description

This task requires writing a Lean 4 method that locates the first occurrence of a specified integer within a sorted array of integers. The method returns the index corresponding to the first time the target value appears in the array; if the target is absent, it returns -1. It is also essential that the original array remains unchanged.

Input

The input consists of:

- `arr`: An array of integers sorted in non-decreasing order.
- `target`: An integer representing the value to search for.

Output

The output is an integer:

- If the target is found, the method returns the index of its first occurrence.
- If the target is not found, the method returns -1.

Note

- The input array must be sorted in non-decreasing order.
- The array is guaranteed to remain unmodified after the method executes.

Verina's Spec

```
def findFirstOccurrence_precond (arr : Array Int) (target : Int) : Prop :=
  -- !benchmark @start precondition
  List.Pairwise (· ≤ ·) arr.toList

-- !benchmark @end precondition

def findFirstOccurrence_postcond (arr : Array Int) (target : Int) (result : Int)
↪ (h_precond : findFirstOccurrence_precond (arr) (target)) :=
  -- !benchmark @start postcondition
  (result ≥ 0 →
    arr[result.toNat]! = target ∧
    (∀ i : Nat, i < result.toNat → arr[i]! ≠ target)) ∧
  (result = -1 →
    (∀ i : Nat, i < arr.size → arr[i]! ≠ target))

-- !benchmark @end postcondition
```

Observation

The post-condition appears to be underspecified.

This is a rather subtle case. When the input is `arr = []` and `target = 0`, the post-condition accepts both 0 and `-1` as valid results, whereas only `-1` is the intended outcome.

This happens because, when `result = 0`, Verina's post-condition requires that

```
(result ≥ 0 →
  arr[result.toNat]! = target ∧
  (∀ i : Nat, i < result.toNat → arr[i]! ≠ target))
```

Since the array is empty, accessing `arr[result.toNat]!` results in an out-of-bounds access. In Lean, such an access returns a default value, namely 0, which accidentally satisfies the post-condition and causes the incorrect result to be accepted.

A possible correction would be to add an explicit bounds check:

```
(result ≥ 0 →
  result.toNat < arr.size ∧
  arr[result.toNat]! = target ∧
  (∀ i : Nat, i < result.toNat → arr[i]! ≠ target))
```

Verina_basic_46

Problem Description

Description

This task requires writing a Lean 4 method that finds the last occurrence of a specified element in a sorted array of integers. The method should return the index corresponding to the last occurrence of the element if it is present; if the element is absent, it should return -1. Additionally, the array must remain unchanged after the method is executed.

Input

The input consists of:

- `arr`: A sorted array of integers in non-decreasing order.
- `elem`: An integer whose last occurrence position is to be determined.

Output

The output is an integer: Returns the index of the last occurrence of the specified integer in the array if it exists. Returns -1 if the integer is not found in the array.

Note

The input array is assumed to be sorted in non-decreasing order and remains unchanged by the method.

Verina's Spec

```
def lastPosition_precond (arr : Array Int) (elem : Int) : Prop :=
  -- !benchmark @start precondition
  List.Pairwise (· ≤ ·) arr.toList

-- !benchmark @end precondition

def lastPosition_postcond (arr : Array Int) (elem : Int) (result: Int) (h_precond
↪ : lastPosition_precond (arr) (elem)) :=
  -- !benchmark @start postcond
  (result ≥ 0 →
    arr[result.toNat]! = elem ∧ (arr.toList.drop (result.toNat + 1)).all (· ≠
↪ elem)) ∧
  (result = -1 → arr.toList.all (· ≠ elem))

-- !benchmark @end postcond
```

Observation

The post-condition appears to be underspecified.

This case is analogous to `basic_37`. When the input is `arr = []` and `elem = 0`, the post-condition accepts both 0 and -1 as valid results, whereas only -1 is the intended outcome.

This happens because, when $\text{result} = 0$, Verina's post-condition requires that

```
(result ≥ 0 →
  arr[result.toNat]! = elem ∧ (arr.toList.drop (result.toNat + 1)).all (· ≠
    ↪ elem))
```

Since the array is empty, accessing `arr[result.toNat]!` results in an out-of-bounds access. In Lean, such an access returns a default value, namely 0, which accidentally satisfies the post-condition and causes the incorrect result to be accepted.

A possible correction would be to add an explicit bounds check:

```
(result ≥ 0 →
  result.toNat < arr.size ∧
  arr[result.toNat]! = elem ∧
  (arr.toList.drop (result.toNat + 1)).all (· ≠ elem))
```

Verina_basic_77

Problem Description

Description

This task involves updating an element within a 2-dimensional array. The goal is to modify only a specific inner array by changing one of its elements to a new value while keeping every other element and all other inner arrays unchanged.

Input

The input consists of:

- `arr`: An array of arrays of natural numbers.
- `index1`: A natural number representing the index in the outer array identifying which inner array to modify (0-indexed).
- `index2`: A natural number representing the index within the selected inner array that should be updated (0-indexed).
- `val`: A natural number which is the new value to set at the specified inner index.

Output

The output is an array of arrays of natural numbers that:

- Has the same overall structure as the input.
- Contains all original inner arrays unchanged except for the inner array at position `index1`.
- In the modified inner array, only the element at `index2` is replaced with `val`, while all other elements remain the same.

Note It is assumed that `index1` is a valid index for the outer array and that `index2` is a valid index within the corresponding inner array.

Verina's Spec

```
def modify_array_element_precond (arr : Array (Array Nat)) (index1 : Nat) (index2
↪ : Nat) (val : Nat) : Prop :=
  -- !benchmark @start precondition
  index1 < arr.size ∧
  index2 < (arr[index1!]).size

-- !benchmark @end precondition

def updateInner (a : Array Nat) (idx val : Nat) : Array Nat :=
  a.set! idx val

def modify_array_element_postcond (arr : Array (Array Nat)) (index1 : Nat)
↪ (index2 : Nat) (val : Nat) (result: Array (Array Nat)) (h_precond :
↪ modify_array_element_precond (arr) (index1) (index2) (val)) :=
  -- !benchmark @start postcondition
  (∀ i, i < arr.size → i ≠ index1 → result[i]! = arr[i]!) ∧
  (∀ j, j < (arr[index1!]).size → j ≠ index2 → (result[index1!])[j]! =
  ↪ (arr[index1!])[j]!) ∧
  ((result[index1!])[index2]! = val)

-- !benchmark @end postcondition
```

Observation

The post-condition appears to be underspecified.

The post-condition ensures that elements other than the updated one remain unchanged, but it does not require that the size of the array remain unchanged. As a result, for the input `#[#[0]], 0, 0, 5`, both `#[#[5]]` and `#[#[5], #[5]]` satisfy the post-condition, even though only the former reflects the intended behavior.

Verina_basic_79

Problem Description

Description

Given a nonempty array of integers and a valid index x (with $1 \leq x < \text{array size}$), the task is to identify two key pieces of information. First, determine the maximum value among the first x elements of the array. Second, select an index p within the range $[x, \text{array size})$ that satisfies the following conditions: if there exists an element in the segment starting from index x that is strictly greater than the previously determined maximum, then p should be the index of the first such occurrence; otherwise, p should be set to the last index of the array. The focus of the problem is solely on correctly identifying the maximum and choosing the appropriate index based on these order conditions.

Input

The input consists of:

- a: An array of integers (assumed to be nonempty).
- x: A natural number (Nat) such that $1 \leq x < a.size$.

Output

The output is a pair (m, p) where:

- m is the maximum value among the first x elements of the array.
- p is an index in the array, with $x \leq p < a.size$, determined based on the ordering condition where $a[p]$ is the first element (from index x onward) that is strictly greater than m. If no such element exists, then p is set to $a.size - 1$.

Note

It is assumed that the array a is nonempty and that the parameter x meets the precondition $1 \leq x < a.size$. The function relies on helper functions to compute the maximum among the first x elements and to select the appropriate index p based on the given conditions.

Verina's Spec

```
def onlineMax_precond (a : Array Int) (x : Nat) : Prop :=
  -- !benchmark @start precondition
  a.size > 0 ∧ x > 0 ∧ x < a.size

  -- x must be at least 1 (as stated in description)
  -- !benchmark @end precondition

def findBest (a : Array Int) (x : Nat) (i : Nat) (best : Int) : Int :=
  if i < x then
    let newBest := if a[i]! > best then a[i]! else best
    findBest a x (i + 1) newBest
  else best

def findP (a : Array Int) (x : Nat) (m : Int) (i : Nat) : Nat :=
  if i < a.size then
    if a[i]! > m then i else findP a x m (i + 1)
  else a.size - 1

def onlineMax_postcond (a : Array Int) (x : Nat) (result: Int × Nat) (h_precond :
  ⇔ onlineMax_precond (a) (x)) :=
  -- !benchmark @start postcondition
  let (m, p) := result;
  (x ≤ p ∧ p < a.size) ∧
  (∀ i, i < x → a[i]! ≤ m) ∧
  (∃ i, i < x ∧ a[i]! = m) ∧
  ((p < a.size - 1) → (∀ i, i < p → a[i]! < a[p]!)) ∧
  ((∀ i, x ≤ i → i < a.size → a[i]! ≤ m) → p = a.size - 1)

  -- !benchmark @end postcondition
```

Observation

Verina's post-condition may not correctly capture the intended semantics.

Specifically, it does not appear to properly characterize that $a[p]$ is the *first* element strictly greater than m .

The fourth condition,

```
((p < a.size - 1) → (∀ i, i < p → a[i]! < a[p]!))
```

does not express the intended “first exceeding” semantics: it constrains $a[p]$ relative to all preceding elements (from index 0), rather than relative to m and to the subarray starting at index x .

A possible correction would be to replace this condition with one that explicitly enforces both strict exceedance of m and the minimality of p within the range starting at x :

```
((p < a.size - 1) →  
  (a[p]! > m) ∧  
  (∀ i, x ≤ i ∧ i < p → a[i]! ≤ m))
```

Verina_basic_84

Problem Description

Description You are given an array of integers and a threshold value k . The problem is to create a new array where every element greater than k is replaced with -1 while every other element remains unchanged.

Input The input consists of:

- arr : An array of integers.
- k : An integer used as the threshold for replacement.

Output The output is an array of integers that satisfies the following conditions:

- For every index i , if $arr[i]$ is greater than k , then the returned array at index i is -1 .
- For every index i , if $arr[i]$ is less than or equal to k , then the returned array at index i remains unchanged.

Note It is assumed that the input array may be empty or non-empty, and that k can be any integer. There are no additional preconditions.

Verina's Spec

```
def replace_precond (arr : Array Int) (k : Int) : Prop :=  
  -- !benchmark @start precondition  
  True  
  
  -- !benchmark @end precondition  
  
def replace_loop (oldArr : Array Int) (k : Int) : Nat → Array Int → Array Int
```

```

| i, acc =>
  if i < oldArr.size then
    if (oldArr[i]!) > k then
      replace_loop oldArr k (i+1) (acc.set! i (-1))
    else
      replace_loop oldArr k (i+1) acc
  else
    acc

def replace_postcond (arr : Array Int) (k : Int) (result: Array Int) (h_precond :
↪ replace_precond (arr) (k)) :=
  -- !benchmark @start postcond
  (∀ i : Nat, i < arr.size → (arr[i]! > k → result[i]! = -1)) ∧
  (∀ i : Nat, i < arr.size → (arr[i]! ≤ k → result[i]! = arr[i]!))

  -- !benchmark @end postcond

```

Observation

The post-condition appears to be underspecified.

The post-condition ensures that elements not exceeding the threshold remain unchanged, but it does not require that the size of the result array matches that of the input. As a result, for the input `#[], 1`, both `#[]` and `#[1]` satisfy the post-condition, even though only the former reflects the intended behavior.

Verina_basic_95

Problem Description

Description

This problem involves swapping two elements in an array of integers at specified positions. Given an array and two indices, the task is to exchange these elements so that the element from the first index moves to the second index and vice versa, while all other elements remain unchanged.

Input

The input consists of:

- arr: An array of integers.
- i: An integer representing the first index (0-indexed) whose element is to be swapped.
- j: An integer representing the second index (0-indexed) whose element is to be swapped.

Output

The output is an array of integers which:

- Has the same size as the input array.

- Contains the element originally at index i in position j and the element originally at index j in position i .
- Leaves all other elements unchanged.

Note

It is assumed that both indices i and j are non-negative and within the bounds of the array (i.e., $\text{Int.toNat } i$ and $\text{Int.toNat } j$ are less than arr.size).

Verina's Spec

```
def swap_precond (arr : Array Int) (i : Int) (j : Int) : Prop :=
  -- !benchmark @start precondition
  i ≥ 0 ∧
  j ≥ 0 ∧
  Int.toNat i < arr.size ∧
  Int.toNat j < arr.size

  -- !benchmark @end precondition

def swap_postcond (arr : Array Int) (i : Int) (j : Int) (result: Array Int)
↪ (h_precond : swap_precond (arr) (i) (j)) :=
  -- !benchmark @start postcondition
  (result[Int.toNat i]! = arr[Int.toNat j]!) ∧
  (result[Int.toNat j]! = arr[Int.toNat i]!) ∧
  (∀ (k : Nat), k < arr.size → k ≠ Int.toNat i → k ≠ Int.toNat j →
    ↪ result[k]! = arr[k]!)

  -- !benchmark @end postcondition
```

Observation

The post-condition appears to be underspecified.

The post-condition ensures that only the two specified positions are swapped while all other elements remain unchanged, but it does not require that the size of the result array matches that of the input. As a result, for the input $\#[1, 2]$, $0, 1$, both $\#[2, 1]$ and $\#[2, 1, 3]$ satisfy the post-condition, even though only the former reflects the intended behavior.

Verina__basic__97

Problem Description

Description

This task involves updating an array of integers such that the element at a specified index is set to 60 while all other elements remain unchanged.

Input

The input consists of:

- a: An array of integers.
- j: A natural number representing the index (0-indexed) to update. It is assumed that j is a valid index ($j < a.size$).

Output

The output is an array of integers where:

- The element at index j is set to 60.
- All other elements remain the same as in the input array.

Note

It is assumed that j is a valid index ($0 \leq j < a.size$).

Verina's Spec

```
def TestArrayElements_precond (a : Array Int) (j : Nat) : Prop :=
  -- !benchmark @start precondition
  j < a.size

-- !benchmark @end precondition

def TestArrayElements_postcond (a : Array Int) (j : Nat) (result: Array Int)
↪ (h_precond : TestArrayElements_precond (a) (j)) :=
  -- !benchmark @start postcondition
  (result[j]! = 60) ∧ (∀ k, k < a.size → k ≠ j → result[k]! = a[k]!)

-- !benchmark @end postcondition
```

Observation

The post-condition appears to be underspecified.

The post-condition ensures that the element at index j is updated to 60 while all other elements remain unchanged, but it does not require that the size of the result array matches that of the input. Consequently, for the input array `#[10, 20, 30]` with `j = 1`, both `#[10, 60, 30]` and `#[10, 60, 30, 40]` satisfy the post-condition, even though only the former correctly reflects the intended behavior.

Verina_basic_103

Problem Description

Description

This problem involves updating an array of integers by modifying two specific positions. Specifically, the element at index 4 should be increased by 3, and the element at index 7 should be changed to 516. The goal is to correctly update these positions while leaving the rest of the array unchanged. The description assumes that the array contains at least 8 elements.

Input

The input consists of

- a: An array of integers. The array must contain at least 8 elements.

Output

The output is an array of integers that meets the following criteria:

- The element at index 4 is updated to its original value plus 3.
- The element at index 7 is set to 516.
- All other elements in the array remain the same as in the input array.

Note

It is assumed that the input array has a size of at least 8 elements. Indices are 0-indexed.

Verina's Spec

```
def UpdateElements_precond (a : Array Int) : Prop :=
  -- !benchmark @start precondition
  a.size ≥ 8

  -- !benchmark @end precondition

def UpdateElements_postcond (a : Array Int) (result: Array Int) (h_precond :
  ⇔ UpdateElements_precond (a)) :=
  -- !benchmark @start postcondition
  result[4]! = (a[4]!) + 3 ∧
  result[7]! = 516 ∧
  (∀ i, i < a.size → i ≠ 4 → i ≠ 7 → result[i]! = a[i]!)
```

Observation

Verina's post-condition appears to be underspecified.

The post-condition ensures that the elements at index 4 and index 7 are updated as required while all other elements remain unchanged. However, it does not require that the size of the result array matches that of the input. Consequently, for the input array

#[10, 20, 30, 40, 50, 60, 70, 80],

both

#[10, 20, 30, 40, 53, 60, 70, 516]

and

#[10, 20, 30, 40, 53, 60, 70, 516, 99]

satisfy the post-condition, even though only the first one correctly reflects the intended behavior.